

Continuidad pedagógica 01 — Saberes previos

Programación en Python · Actividad de continuidad pedagógica · Inicio del curso

Datos de referencia

Campo	Detalle
Curso	Programación en Python
Momento de uso	Antes de la Unidad 1 (inicio del curso, previo al encuentro 4)
Duración teórica	120 minutos (2 horas reloj)
Requisitos	Contenidos de nivel secundario básico: lógica de programación en lenguaje natural (algoritmos, secuencias, decisiones), números y operaciones aritméticas, organización de archivos y carpetas en la computadora. No se requiere sintaxis Python. Para las tareas de programación: computadora con terminal del sistema operativo.
Entrega	Individual, manuscrita (parte en papel) + resultados de las tareas en computadora.

Objetivos

1. Recuperar la noción de **algoritmo** como secuencia ordenada de pasos para resolver un problema cotidiano.
2. Identificar **decisiones** simples y compuestas en lenguaje natural, reconociendo las condiciones que determinan cada camino.
3. Afianzar las **operaciones aritméticas básicas** y la prioridad de operadores.
4. Reconocer la **organización de archivos y carpetas** como un sistema de árbol que permite localizar y almacenar información.

Actividades

#	Actividad	Puntaje	Tiempo estimado
1	Algoritmo cotidiano (papel). Escribí, en lenguaje natural y paso a paso, el algoritmo para retirar dinero de un cajero automático. Identificá claramente el inicio, los pasos intermedios y el final.	20	25 min
2	Decisiones en la vida diaria (papel). Leé cada situación y respondé escribiendo la condición y la decisión que tomarías: (a) salir con paraguas, (b) elegir transporte para llegar a horario, (c) decidir si un número es par o impar con una explicación.	20	25 min

#	Actividad	Puntaje	Tiempo estimado
3	Operaciones y prioridad (papel). Resolvé las siguientes expresiones indicando el resultado final y el orden en que se evalúan las operaciones: (a) $2 + 3 \times 4$, (b) $(2 + 3) \times 4$, (c) $10 - 4 \div 2 + 1$, (d) $8 \div 2 \times (2 + 2)$. Explicá con tus palabras qué regla determina el orden.	20	20 min
4	Árbol de directorios (papel). Dado el siguiente árbol de carpetas, respondé en tu hoja: (a) ¿Cuántas carpetas contiene <code>Documentos/</code> ? (b) ¿Cuál es la ruta desde <code>Inicio</code> hasta <code>foto.jpg</code> ? (c) Si estás en <code>Música/</code> , ¿cómo llegás a <code>receta.txt</code> sin volver a <code>Inicio</code> ? (d) ¿Qué archivos están dentro de <code>Trabajos/</code> ?	20	20 min
5	Tarea en computadora: estructura de carpetas. Abrí la terminal de tu computadora y creá, usando solo comandos (<code>mkdir</code> , <code>cd</code> , <code>ls</code> o <code>dir</code>), la siguiente estructura de carpetas para tu cursada: <code>python-curso/unidad-1/</code> , <code>python-curso/unidad-2/</code> , <code>python-curso/unidad-3/</code> , <code>python-curso/trabajo-final/</code> . Luego, dentro de <code>unidad-1/</code> , creá una carpeta <code>ejercicios</code> y otra <code>tp</code> . Verificá con <code>ls</code> o <code>dir</code> que la estructura quedó correcta y anotá los comandos que usaste.	20	30 min
Total		100	120 min

Autoevaluación

Respondé con sinceridad. No hay nota correcta o incorrecta; esto te sirve a vos para saber en qué prestar atención durante la Unidad 1.

1. **Antes de estas actividades, ¿podías explicar qué es un algoritmo con tus palabras?** Sí / Más o menos / No.

Reflexión breve: ¿cambió tu idea después de la Actividad 1?

2. **Cuando te enfrentás a una decisión cotidiana (ej.: qué comer), ¿identificás mentalmente las condiciones que evaluás?** Siempre / A veces / Nunca.

Reflexión breve: ¿te resultó fácil escribir las condiciones de la Actividad 2?

3. **¿Recordabas la prioridad de las operaciones (paréntesis, multiplicación/división, suma/resta)?** Sí, bien / A medias / No, lo había olvidado.

Reflexión breve: ¿en qué ejercicio de la Actividad 3 tuviste más dudas?

4. **¿Usaste la terminal antes de esta actividad?** Nunca / Algunas veces / Con frecuencia.

Reflexión breve: ¿cómo te sentiste creando carpetas con comandos en lugar de usar el mouse?

5. **¿Qué concepto de los repasados te gustaría practicar más antes de arrancar con Python?**

Escribí uno o dos temas.

Nota académica

La resolución se realiza en forma habitual (por lo general, en grupo); las tareas de programación requieren el uso de la computadora; la presentación es individual y manuscrita, al inicio de la próxima clase, y constituye una actividad más de la asignatura que forma parte del proceso de evaluación.

Continuidad pedagógica 02 — Tras la evaluación de la Unidad 1

Programación en Python · Actividad de continuidad pedagógica · Tras el encuentro 9

Datos de referencia

Campo	Detalle
Curso	Programación en Python
Momento de uso	Después del encuentro 9 (evaluación de la Unidad 1)
Duración teórica	120 minutos (2 horas reloj)
Requisitos	Contenidos de la Unidad 1: variables y tipos (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>), entrada y salida (<code>input()</code> / <code>print()</code> , conversiones de tipo), condicionales (<code>if</code> / <code>elif</code> / <code>else</code>), operadores relacionales y lógicos, bucles <code>for</code> y <code>while</code> con <code>range()</code> , contadores y acumuladores. Para las tareas de programación: computadora con Python y VS Code.
Entrega	Individual, manuscrita (parte en papel) + archivos <code>.py</code> de las tareas de programación.

Objetivos

1. Consolidar el uso de **variables y tipos** en la escritura y lectura de código Python.
2. Resolver ejercicios de **condicionales** que requieran evaluar condiciones múltiples con `if` / `elif` / `else` .
3. Aplicar **bucles** (`for` y `while`) con contadores y acumuladores en problemas de mediana complejidad.
4. Escribir un programa completo desde cero que integre entrada, proceso condicional y repetición.

Actividades

#	Actividad	Puntaje	Tiempo estimado
1	Completar código: declaración y tipos (papel). Dado un programa incompleto, escribí las variables faltantes con el tipo adecuado (<code>int</code> , <code>float</code> , <code>str</code> , <code>bool</code>) y completá las conversiones de tipo donde haga falta.	15	15 min
2	Predicción de salida: condicionales (papel). Leé cada fragmento de código con <code>if</code> / <code>elif</code> / <code>else</code> y escribí exactamente lo que muestra la consola. Explicá brevemente por qué.	15	20 min
3	Completar bucle: contadores y acumuladores (papel). Dado un programa con un bucle <code>for</code> o <code>while</code> incompleto, completá la línea que cuenta o acumula según la consigna. Indicá qué valor muestra el <code>print()</code> final.	20	25 min
4	Hallar el error (papel). Cada fragmento contiene uno o dos errores (sintácticos o lógicos). Subrayalos, explicá por qué están mal y escribí la versión corregida.	15	20 min
5	Tarea de programación: validador de notas. Escribí un programa <code>validador_notas.py</code> que: (a) pida al usuario 5 notas una por una usando un bucle, (b) valide que cada nota esté entre 1 y 10 usando un condicional, (c) si la nota es inválida, vuelva a pedirla, (d) al final muestre el promedio y un mensaje "APROBADO" si el promedio ≥ 6 , "DESAPROBADO" si es menor. Usá un contador y un acumulador. Comentá el código.	35	40 min
Total		100	120 min

Autoevaluación

1. ¿Podés explicar la diferencia entre `int` y `float` sin mirar apuntes? Sí / Más o menos / No.

Reflexión breve: ¿en qué actividad tuviste que usar esa diferencia?

2. Cuando escribís un `if`, ¿siempre considerás qué pasa si ninguna condición se cumple (el `else`)? Siempre / A

veces / Casi nunca.

Reflexión breve: ¿te saltaste algún caso en la Actividad 2 o 4?

3. **¿Sabés cuándo conviene usar `for` en lugar de `while` y viceversa?** Sí, claro / Tengo dudas / No.

Reflexión breve: ¿qué criterio usaste para elegir en la Actividad 5?

4. **¿Recordás la sintaxis de `range()` para generar secuencias?** Sí / A veces confundo los argumentos / No.

Reflexión breve: ¿en qué ejercicio te trabaste con `range()` ?

5. **¿Qué tema de la Unidad 1 sentís que necesitás repasar más?**

Escribí uno o dos temas.

Nota académica

La resolución se realiza en forma habitual (por lo general, en grupo); las tareas de programación requieren el uso de la computadora; la presentación es individual y manuscrita, al inicio de la próxima clase, y constituye una actividad más de la asignatura que forma parte del proceso de evaluación.

Continuidad pedagógica 03 — Tras la evaluación de la Unidad 2

Programación en Python · Actividad de continuidad pedagógica · Tras el encuentro 15

Datos de referencia

Campo	Detalle
Curso	Programación en Python
Momento de uso	Después del encuentro 15 (evaluación de la Unidad 2)
Duración teórica	120 minutos (2 horas reloj)
Requisitos	Contenidos de las Unidades 1 y 2: variables y tipos, entrada/salida, condicionales, bucles, contadores y acumuladores (U1); listas y métodos, mutabilidad, tuplas, conjuntos (<code>set</code>), diccionarios, funciones (<code>def</code> , <code>return</code> , ámbito local), bloque <code>if __name__ == "__main__"</code> (U2). Para las tareas de programación: computadora con Python y VS Code.
Entrega	Individual, manuscrita (parte en papel) + archivos <code>.py</code> de las tareas de programación.

Objetivos

1. Manipular **listas** con sus métodos principales (`append` , `remove` , `pop` , `sort` , `index`) en ejercicios de código.
2. Distinguir **tuplas**, **conjuntos** y **diccionarios** según su uso y propiedades (mutabilidad, unicidad, clave↔valor).
3. Escribir **funciones** con `def` y `return` , diferenciando variables locales y globales.
4. Integrar estructuras de datos y funciones en un programa completo con `if __name__ == "__main__"` .

Actividades

#	Actividad	Puntaje	Tiempo estimado
1	Completar código: métodos de listas (papel). Dado un programa que opera sobre una lista de números, completá las líneas usando los métodos apropiados (<code>append</code> , <code>remove</code> , <code>pop</code> , <code>sort</code> , <code>index</code>). Indicá qué contiene la lista al final.	15	20 min
2	Elección de estructura (papel). Para cada situación, elegí y justificá si conviene usar lista, tupla, conjunto o diccionario: (a) almacenar los días de la semana en orden, (b) guardar los nombres de estudiantes sin repetidos, (c) asociar un nombre de producto con su precio, (d) registrar temperaturas horarias de un día que no van a cambiar.	15	20 min
3	Escribir una función (papel). Escribí el código de una función <code>calcular_promedio(lista_numeros)</code> que recibe una lista de <code>float</code> y devuelve el promedio. Incluí el <code>if __name__ == "__main__"</code> con un ejemplo de uso que cree una lista, llame a la función y muestre el resultado.	20	25 min
4	Hallar el error: ámbito y tipos (papel). Cada fragmento tiene un error de ámbito (variable local usada como global) o de tipo (modificar una tupla, usar un conjunto con índice). Señalalo, explicá por qué está mal y corregilo.	15	15 min
5	Tarea de programación: gestor de contactos. Escribí un programa <code>gestor_contactos.py</code> que: (a) use un diccionario donde cada clave es un nombre (<code>str</code>) y cada valor es un teléfono (<code>str</code>), (b) tenga una función <code>agregar_contacto(directorio, nombre, telefono)</code> que agrega un nuevo par, (c) tenga una función <code>listar_contactos(directorio)</code> que recorre el diccionario y muestra cada contacto, (d) en el bloque <code>if __name__</code> cree un directorio vacío, agregue al menos 3 contactos y los liste. Incluí comentarios.	35	40 min
Total		100	120 min

Autoevaluación

1. **¿Podés explicar la diferencia entre una lista y una tupla sin dudar?** Sí / Más o menos / No.

Reflexión breve: ¿en qué actividad usaste esa diferencia?

2. **¿Cuándo usarías un conjunto (`set`) en lugar de una lista?** Lo tengo claro / Tengo dudas / No lo sé.

Reflexión breve: ¿qué propiedad del conjunto lo hace útil para evitar duplicados?

3. **En tus funciones, ¿diferenciás bien las variables locales de las globales?** Siempre / A veces / Casi nunca.

Reflexión breve: ¿tuviste algún error de ámbito en la Actividad 4?

4. **¿Recordás para qué sirve el bloque `if __name__ == "__main__"` ?** Sí, lo explico / Lo uso pero no sé explicarlo / No.

Reflexión breve: ¿lo usaste en la Actividad 5?

5. **¿Qué estructura de datos (lista, tupla, conjunto o diccionario) te resultó más difícil de entender y por qué?**

Escribí tu respuesta brevemente.

Nota académica

La resolución se realiza en forma habitual (por lo general, en grupo); las tareas de programación requieren el uso de la computadora; la presentación es individual y manuscrita, al inicio de la próxima clase, y constituye una actividad más de la asignatura que forma parte del proceso de evaluación.

Continuidad pedagógica 04 — Tras la evaluación de la Unidad 3

Programación en Python · Actividad de continuidad pedagógica · Tras el encuentro 26

Datos de referencia

Campo	Detalle
Curso	Programación en Python
Momento de uso	Después del encuentro 26 (evaluación de la Unidad 3)
Duración teórica	120 minutos (2 horas reloj)
Requisitos	Contenidos de las Unidades 1 a 3: variables, entrada/salida, condicionales, bucles (U1); listas, tuplas, conjuntos, diccionarios, funciones, <code>if __name__</code> (U2); métodos de cadenas (<code>split</code> , <code>strip</code> , <code>join</code> , <code>replace</code>), f-strings con formato, validación con <code>try / except ValueError</code> y <code>while True</code> , menú de consola en memoria (U3). Para las tareas de programación: computadora con Python y VS Code.
Entrega	Individual, manuscrita (parte en papel) + archivos <code>.py</code> de las tareas de programación.

Objetivos

1. Aplicar **métodos de cadenas** (`split` , `strip` , `join` , `replace`) para procesar texto ingresado por el usuario.
2. Usar **f-strings con formato** (`f"{valor:.2f}"` , alineación) para presentar información tabulada.
3. Implementar **validación robusta** con `try / except ValueError` y bucles `while True` para entrada de datos.
4. Construir un **menú de consola** que ejecute opciones hasta que el usuario decida salir.

Actividades

#	Actividad	Puntaje	Tiempo estimado
1	Completar código: métodos de cadenas (papel). Dado un programa que recibe una frase y debe separarla en palabras, limpiar espacios y reemplazar caracteres, completá las líneas con los métodos correctos (<code>split</code> , <code>strip</code> , <code>join</code> , <code>replace</code>).	10	15 min

#	Actividad	Puntaje	Tiempo estimado
2	Predecir salida: f-strings con formato (papel). Lee cada fragmento con f-strings que usan ancho de campo, cantidad de decimales o alineación y escribí exactamente lo que se muestra en consola.	10	15 min
3	Escribir validación (papel). Escribí el código de una función <code>pedir_entero(mensaje)</code> que recibe un texto, lo muestra como prompt, y usa un bucle <code>while True</code> con <code>try / except ValueError</code> para asegurarse de devolver un <code>int</code> . Incluí un ejemplo de uso que llame a la función y muestre el doble del valor ingresado.	20	25 min
4	Completar un menú (papel). Dado un programa con un menú esqueleto (<code>while True</code> , <code>input</code> de opción, <code>if / elif / else</code>), completá las opciones faltantes (agregar ítem, listar ítems, salir). El menú trabaja sobre una lista en memoria.	20	20 min
5	Tarea de programación: analizador de texto con menú. Escribí un programa <code>analizador_texto.py</code> que presente un menú con estas opciones usando un diccionario en memoria como almacenamiento: (1) Ingresar frase — pide una frase y la guarda asociada a un título; (2) Mostrar palabras — usa <code>split</code> para mostrar cada palabra separada; (3) Contar caracteres — cuenta letras, espacios y signos; (4) Salir. Cada opción debe estar validada con <code>try / except</code> si corresponde y los resultados deben mostrarse con f-strings formateadas. Incluí comentarios.	40	45 min
Total		100	120 min

Autoevaluación

1. ¿Recordás qué hace `split()` y qué devuelve? Sí / Más o menos / No.

Reflexión breve: ¿en qué actividad lo usaste o identificaste?

2. ¿Sabés cómo mostrar un número con solo dos decimales en un f-string? Sí, lo uso / Lo vi pero no lo recuerdo / No.

Reflexión breve: ¿para qué serviría en un programa real?

3. ¿Entendés por qué `try / except` evita que el programa se detenga con un error? Sí, claro / A medias / No.

Reflexión breve: ¿qué pasa si no usás validación y el usuario ingresa texto donde se espera un número?

4. Cuando armás un menú con `while True`, ¿cómo hacés para que el programa termine cuando el usuario elige “Salir”? Lo explico / Tengo una idea / No sé.

Reflexión breve: ¿usaste `break`? ¿dónde lo pondrías?

5. De todos los temas de las Unidades 1 a 3, ¿cuál te gustaría reforzar antes del Trabajo Final?

Escribí uno o dos temas.

Nota académica

La resolución se realiza en forma habitual (por lo general, en grupo); las tareas de programación requieren el uso de la computadora; la presentación es individual y manuscrita, al inicio de la próxima clase, y constituye una actividad más de la asignatura que forma parte del proceso de evaluación.